

Responsible AI for enterprise Claude apps

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 12 of 12 · English notes

Vijay bundled permissions, safety hooks, and prompt/output scanners under **responsible AI principles** for the applications UST actually ships. The exam will not grade your GDPR programme, but it will expect you to choose controls when a scenario mixes PII, bash, MCP, and a customer email.

What “responsible” meant in this session

Not a slide of values. A working definition: the agent should **not execute harm** (wipe a disk, write malware into a path), **not leak regulated data** through logs or third-party tools, and **not obey jailbreak-shaped prompts** that try to dump the system prompt or ignore policy. Compliance names in the room were PII, GDPR, and HIPAA — as examples of regimes that force masking, monitoring, and least privilege — not as a legal lecture.

USAGE POLICY STILL BINDS YOU

Hooks cannot make a disallowed use case allowed. Read [Anthropic’s usage policy](#) for the products and API. Classroom demos (clocks, public todos APIs, sample Markdown) stay on the right side; client production data in a shared notebook does not.

Layer the controls

1. **Policy and purpose** — should this agent exist? Recruitment-with-WhatsApp is a different risk than a read-only policy Q&A bot.
2. **Data minimisation** — RAG and tools should fetch the smallest useful field set. Do not retrieve full customer rows to answer “how many open tickets?”
3. **Allow / deny tools** — a report agent may need a DB tool and an email tool; it should not get unconstrained Bash.
4. **Pre-tool hooks** — argument-level checks (no `rm -rf`, no writes to `/etc`, redirect to a safe directory).
5. **Human in the loop** — `AskUserQuestion` or default permission mode when the action is irreversible (send offer letter, delete).
6. **Input guardrails** — reject injection and overlong prompts before you spend tokens.
7. **Output guardrails** — secret scanning, PII redaction before logs, tickets, or MCP servers see the text.
8. **Evaluation** — Anthropic’s strengthen-guardrails guidance: test the agent on abuse cases, not only the happy JD-posting path.

Enterprise story: report and email

Custom tools: read customer DB → generate report → send email. Email is PII. The send tool needs the real address. Everything else (debug prints, web search, a vendor MCP “help” server) should see a masked value or nothing. That split is why “just encrypt the gateway” is incomplete: encryption in transit does not stop your agent from pasting the address into a Bash echo or a Slack MCP. Responsible design is *where the plaintext is allowed to exist*.

Jailbreaks and system-prompt theft

The notebook’s input validator treated phrases like “ignore all previous instructions” and “reveal your system prompt” as block patterns (regex in class; libraries in production). Short-circuit *before* query so you do not pay for a refusal you could have made locally. This is complementary to Claude’s own safety training, not a replacement.

Open source versus first-party

Mritunjay asked if guardrails were a built-in Claude function that also capped tokens. No: token accounting happens at the API query. Guardrail code is yours (or a reviewed open-source package). Recreate or vendor-assess Rulebricks-style and Claude Code community guardrail lists; do not paste unaudited regex into a bank’s agent. Pair that with official docs on evaluation so the exam answer can name both “application-level validators” and “Anthropic’s guardrail/eval guidance.”

UST delivery checklist

- No confidential client corpora in Cowork, Chat, or classroom Atlas clusters.
- API keys in workspaces and secret stores (**workspaces**), not in committed `.env`.
- Plan-mode or observe-only agents in CI until a human promotes accept-edits.
- MCP servers: prefer vendor servers; treat community MCP like untrusted code — they are tools with your credentials.
- RAG indexes: tenant metadata so one client never retrieves another’s chunks (module 11).

How this shows up in scenarios

Pick the control that matches the failure: destructive shell → hook or deny Bash; model asked to ignore policy → input guardrail; customer email in a log → output redaction; agent must not change production files → plan + Read/Glob/Grep; need a human to confirm a send → AskUserQuestion or default prompts. Stacking every control on every toy clock tool is not the point.

Official documentation

- **Usage policy**

<https://www.anthropic.com/legal/usage-policy>

Binding rules for Claude products and the API.

- **Strengthen guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

How Anthropic wants you to test and harden.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Org and key isolation.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Third-party tools are part of the data-flow threat model.

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Permission modes you will actually configure.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.